

FraudShield

Real-time transaction fraud detection where the scoring engine is Exasol itself — every rule runs as a live SQL analytic query, the instant a transaction lands.

01

Exasol — the analytics engine at the core

02

FastAPI ingestion layer

03

Transaction simulator

04

Streamlit live ops dashboard

Fraud rules usually live far away from the data they judge

§ 01

T+batch

Rules run in overnight jobs or a separate feature pipeline, long after the transaction has already settled.

app-code rules

Fraud logic gets hardcoded into services, so every new pattern is a deploy, not a query.

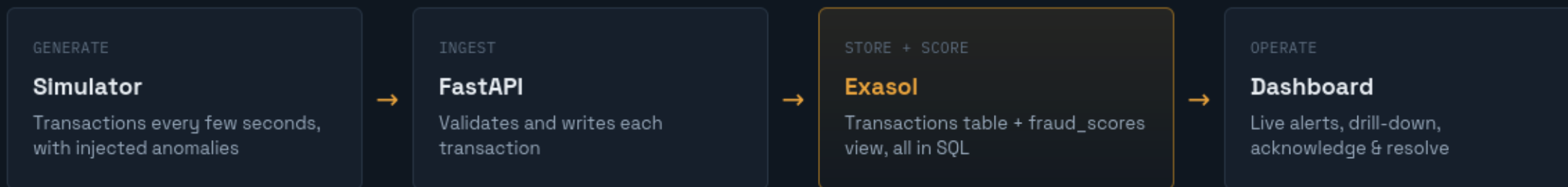
no ops loop

Alerts land in a log file, not a workflow — nobody owns acknowledging or resolving them.

FraudShield's answer is to put the scoring logic where the data already lives: inside Exasol, as SQL. That's what makes it real-time.

One data path, one source of truth

§ 02



Everything downstream of ingestion — scoring, history, profiles, alert state — is just a query against Exasol. There's no separate feature store, cache, or batch job to keep in sync.

Exasol isn't storage here — it's the fraud engine

§ 03

Scoring is SQL, not a service

Every rule — spend spikes, velocity, geo-sequence, new merchants — is a view defined in `fraud_views.sql`. Tuning a threshold means editing SQL, not redeploying an app.

Built for exactly this workload

Exasol is a columnar, in-memory MPP database designed for fast analytical aggregation — the same shape of query a rolling risk score needs, run against every incoming row.

Analytic SQL replaces a feature pipeline

Window functions compute a customer's rolling average, a one-minute transaction count, and a prior-city lookup directly over the transactions table — no Spark job, no offline feature store.

One database, one truth

Transactions, risk scores, customer/merchant profiles, and alert lifecycle state all live in Exasol. The dashboard and API are thin clients over the same data.

Four rules, expressed as one Exasol view

§ 04

fraud_views.sql

Exasol SQL

```
CREATE OR REPLACE VIEW fraud_scores AS
SELECT t.*,
  AVG(amount) OVER (PARTITION BY customer_id) AS avg_amount,
  COUNT(*) OVER (PARTITION BY customer_id
    ORDER BY txn_time RANGE BETWEEN 60 PRECEDING AND CURRENT ROW) AS txns_1m,
  LAG(city) OVER (PARTITION BY customer_id ORDER BY txn_time) AS prev_city,
  LAG(txn_time) OVER (PARTITION BY customer_id ORDER BY txn_time) AS prev_time
FROM transactions t; -- risk_score + classification computed from these columns
```

Window functions do the work app code usually does: rolling averages, time-bucketed counts, and prior-row lookups, all in one pass over the table.

The view is live. Every read reflects the latest transactions — the dashboard just selects from fraud_scores, no recomputation step.

New rule = new column. Extending detection means adding a column expression to the view, not shipping a new service.

The rules that view computes

§ 05

+40 Transaction amount exceeds 5× the customer's historical average

+20 More than five transactions from the same customer within one minute

+30 Chennai transaction followed by a Delhi transaction within 15 minutes

+10 Merchant has never been used by this customer before

SAFE

0–30

REVIEW

31–60

FRAUD

61–100

Everything an analyst needs, one screen

§ 06

01

Live alert table

Auto-refreshes every 10 seconds as Exasol scores new transactions

02

City risk & alert velocity

Geographic and time-based views of where risk is concentrating

03

Rule contribution & merchant concentration

Shows which rules are firing and which merchants are driving alerts

04

Alert drill-down

Full customer transaction history alongside the flagged event

05

Customer & merchant risk profiles

Transaction count, average amount, alert count, cities, and merchants

06

Spend anomaly views

Surfaces amount and behavior outliers alongside rule-based alerts

Every alert has an owner and a channel

§ 07



Lifecycle state is persisted back into Exasol, so operators can acknowledge or resolve alerts from the dashboard. Transactions scoring above 60 are also pushed out automatically:

WEBHOOK

Posts the alert payload to any configured HTTP endpoint

SLACK

Sends a formatted alert message to a configured channel

EMAIL (SMTP)

Delivers alert details to configured recipients

The fraud engine is a SQL view, not a service.

FraudShield puts the analytics inside Exasol, where the transaction data already lives — so detection happens the instant a row is written, and a new rule is a line of SQL away.

ENGINE	RULE FILE	SCORES	OPS
Exasol analytic SQL	fraud_views.sql	0-100, three tiers	Live dashboard + alerts